

Draw It or Lose It

CS 230 Project Software Design

Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 5
Recommendations Error! Bookmark not defined.

Document Revision History

Version	Date	Author	Comments
1.0	9/20/26	Mahogany Holder	Draft of the Executive Summary and Design Constraints
2.0	10/4/26	Mahogany Holder	Added Evaluation table (server side, client side, development tools)
3.0	10/18/26	Mahogany Holder	Added Recommendations

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Currently, The Gaming Room offers Draw It or Lose It, which is available only as an Android application. They would like to move to a web-based version that allows them to serve more users across multiple platforms. The issue the company faces is that they lack the expertise needed to establish this type of development environment and design a distributed system architecture that allows them to reach users across different devices.

They have outsourced the project to Creative Technology Solutions. After reviewing the pros and cons of multiple operating systems, they are recommending developing Draw It or Lose It as a web-based distributed application hosted on a Linux server. With this approach, a single centralized application will be able to serve users on Linux, Mac, Windows, and mobile devices through a web browser. This will also eliminate the need to create multiple native applications for each platform.
The application will be designed using object-oriented design principles and software design patterns, specifically using the Singleton pattern, where only one instance of the GameService exists in memory, and the Iterator pattern, which can be used to iterate through existing games, teams, and players when checking for duplicate names.

This approach will allow the server infrastructure to scale up or down based on demand. It will also allow security to be built into the application from the beginning of development through production. Additionally, the application will be hosted in an environment that supports easier maintenance and will allow developers to build functionality that can gracefully handle connectivity interruptions and invalid data being entered into the system.

Requirements

1. A game will have the ability to have one or more teams involved.
2. Each team will have multiple players assigned to it.
3. Game and team names must be unique to allow users to check whether a name is in use when choosing a team name.
4. Only one instance of the game can exist in memory at any given time. This can be accomplished by creating unique identifiers for each instance of a game, team, or player.

Design Constraints

Scalability

- As interest in the game grows or declines, the design must be able to handle changes in the number of requests to the server. The application should be prepared to use load balancing, efficient database queries, and scalable cloud resources so that it can support increases or decreases in demand without negatively affecting performance.

Data Consistency

- It is important to implement a design that can validate and synchronize information when multiple players are interacting with the same game objects or data at the same time. This helps ensure that players receive the correct information and current game state and prevents conflicting or outdated data from being stored.

Cross-Platform Compatibility

- For a single application to serve users across multiple platforms, it is important to account for different screen sizes, browsers, and input methods. The application should use standard browser-based web technologies and responsive design so that game elements, drawings, timers, and other interfaces render correctly regardless of the browser, device, or screen size being used.

Network Connectivity

- Communication between the user's device and the server is important with a distributed web application. The design should include error handling, reconnection functionality, and appropriate validation to prevent incomplete data from negatively affecting the game.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

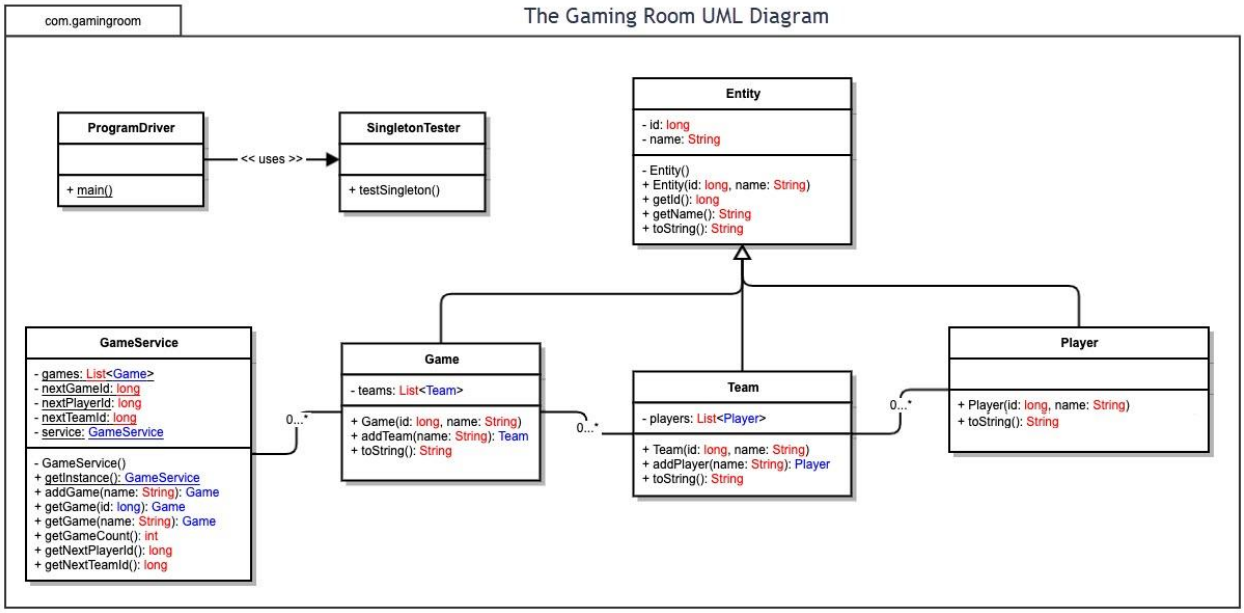
Domain Model

We see that there is an open triangle connecting three classes: Game, Team, and Player. This relationship represents inheritance, meaning they inherit attributes and methods from the Entity class. We also see relationships between the classes. A Team can have zero to many Players, and a Game can have zero to many Teams. The GameService class also has a zero-to-many relationship with Game because it contains a List<Game> and can manage multiple games.

Through these relationships, we see several object-oriented programming principles being demonstrated in the diagram. First, we see encapsulation. This is shown by the dashes, which represent private attributes, and the plus signs, which represent public methods. This means that some attributes are not directly accessible from a separate class, which helps protect the data. The public methods allow data to be accessed from the class through methods such as getters.

We also see inheritance with the Game, Team, and Player classes inheriting from the Entity class. Polymorphism is also demonstrated because these classes can be treated as Entity objects while still having their own attributes and methods. We see this with the toString methods within each Game, Team, and Player class.

The diagram also shows the Singleton design pattern being used with the GameService class. This is demonstrated through the getInstance method, which indicates that the application is designed to have only one instance of GameService. This single service is responsible for managing the games overall. These design patterns and structures will help us meet the software requirements while allowing the design to be efficient, reusable, protect the data, and properly manage the relationships between classes.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	Mac computers can host a web-based software application and provide a stable and secure operating environment. However, macOS is not as commonly used as a production web servers. One benefit of using a Mac is that developers can build and test software in a Unix-based environment. macOS is based on Unix technologies, which makes it compatible with many development tools and technologies that are also commonly used in Linux environments. This can make macOS useful during the development and testing stages of an application. However, there are many disadvantages to using Mac hardware for a production server. Apple hardware can be more expensive than some Windows or Linux server hardware, and macOS is designed primarily for desktop and workstation use rather than large-scale server deployment. In addition, macOS is licensed by Apple for use on Apple hardware, which limits the hardware options available for deploying a macOS-based server. For The Gaming Room, this would not be the best choice for hosting the web-based application on a production server.	Linux is commonly used for hosting web applications. It has a large ecosystem of resources and support for web servers, databases, programming languages, and development frameworks. For example, web-server software such as Nginx and Apache can be used on Linux to receive and process web requests. Another advantage of Linux is its stability, flexibility, and low licensing costs. Many Linux distributions, including Ubuntu and Debian, are available at no cost and can be used without purchasing an operating-system license. However, using Linux may require more command-line and system-administration knowledge than operating systems that provide more graphical management tools. Organizations may also need to pay for professional support, such as the support services offered by companies such as Red Hat or SUSE. Linux provides options for scaling a web application as the number of users increases. Linux would be a good choice for hosting the web-based application on the production server.	Windows can host a web-based software application, and many organizations are familiar with its graphical environment. It has a strong base of resources and support for development tools, databases, enterprise applications, and security features. Windows Server can use Internet Information Services (IIS) as its web server. IIS receives and manages web requests. However, Windows Server can have higher licensing costs because organizations typically need to purchase licenses to use it. The exact licensing cost depends on factors such as the Windows Server edition, the number of processor cores, and the number of users or devices accessing services, depending on the licensing model being used. For The Gaming Room, Windows could work for hosting the web-based application on the production server, but it would likely be more expensive than other options.	Mobile devices are designed for a single user and generally are not used to host web applications. They are not designed with the same server software or tools needed to manage continuous background services. Mobile devices also have limited resources, such as memory and processing power, so they would not be able to reliably handle thousands of requests and updates. Generally, smartphones and tablets act as clients that connect to a web-based application hosted on a server.

Client Side	Supporting Mac clients would require the application to work properly with common Mac browsers such as Safari, Chrome, and Firefox. Since the game is web-based, users would not necessarily need a separate Mac application. The main development considerations would be Safari/WebKit testing, user interface testing, browser compatibility, and how images, timers, WebSockets for real-time synchronization, and other game features display on macOS. Developers would also need to ensure that the application performs consistently in Safari, since Safari may handle certain web technologies differently from Chrome or Firefox. The main costs would involve additional testing time, developers who understand cross-platform web development, and access to Apple devices or a cloud testing service to test Safari.	Supporting Linux clients would require the application to work properly with common browsers such as Firefox and Chrome-based browsers. Since the game is web-based, users would not necessarily need a separate Linux application. Developers would need to test the game using browsers commonly available on Linux, such as Firefox and Chrome-based browsers. Another consideration is that Linux users may use different distributions and desktop environments, which can create additional compatibility and testing requirements. The main costs would involve additional testing time across browsers and distributions and developers who understand cross-platform web development. Free tools such as Firefox and Chrome can help keep testing costs low. Using standard web technologies would make it easier to support Linux clients.	Supporting Windows clients would require the application to work properly with common browsers such as Microsoft Edge, Chrome, and Firefox. Since the game is web-based, users would not necessarily need a separate Windows application. Another consideration would be the wide variety of Windows hardware and screen sizes, along with how SVG graphics render. Developers would need to ensure that the application performs properly across different hardware configurations and display resolutions. The main costs would involve additional testing time across different hardware and display configurations and developers who understand cross-platform web development and Windows-based environments. Automated testing could reduce the time needed for repeated testing.	Mobile devices would allow players to access the game from smartphones or tablets, making the application more flexible for users. The main consideration would be creating a responsive interface using CSS media queries and Flexbox/Grid that works with different screen sizes, operating systems, and browsers. Developers would need to test on many versions of Android and iOS devices. They would also need to consider touchscreen controls, portrait and landscape orientations, smaller screens, mobile performance, battery usage, and varying network conditions. Supporting mobile devices would require more testing time and expertise in responsive web design. Costs would include testing on a range of Android and iOS devices, with physical devices. Building a responsive web application versus a native application helps avoids the app store fees and review process.

Development Tools	Developers can use tools such as Visual Studio Code, IntelliJ IDEA, Homebrew or Docker Desktop on Mac. However, Xcode and Safari WebKit development tools are only available on Mac. Some tools are free, while others have paid versions, such as IntelliJ IDEA Ultimate. Common programming languages for the web application could include Java, JavaScript, HTML, CSS, and SQL. Java could be used for the application logic, while JavaScript, HTML, and CSS could be used to create the client-side interface. SQL could be used to manage and retrieve data from a database. Git could also be used for source control. Since macOS is Unix-based, developers can also use many command-line tools and development environments that are commonly used on Linux servers. A separate development team would not be needed for Mac because the application will not use Xcode or Swift to build a native iOS app. The application will be running in web browsers, so developers who understand cross-platform web development can work on the application using shared languages and tools.	Developers can use tools such as Visual Studio Code, IntelliJ or Docker Engine on Linux. Some professional versions have to be paid for, such as IntelliJ IDEA Ultimate, while others, such as Visual Studio Code and IntelliJ IDEA Community Edition, are free. Common programming languages for the web application could include Java, JavaScript, HTML, CSS, and SQL. Java could be used for the application logic, while Maven or Gradle could be used to manage Java dependencies and build the application. JavaScript, HTML, and CSS could be used to create the client-side interface, while SQL could be used to manage and retrieve data from a database. Git could also be used for source control. Linux also provides support for command-line development tools, which can be useful for building, testing, and deploying web applications. Linux does not require a separate development team, and the same team that builds the web application can also work with the Linux server as long as someone has the knowledge to handle.	Developers can use tools such as Visual Studio Code, IntelliJ IDEA, SQL Server, or PowerShell. Some of these tools have to be paid for, such as IntelliJ IDEA Ultimate, Azure DevOps, and Visual Studio Professional. Common programming languages for the web application could include Java, JavaScript, HTML, CSS, and SQL. Java could be used for the application logic, while Maven or Gradle could be used to manage Java dependencies and build the application. JavaScript, HTML, and CSS could be used to create the client-side interface, while SQL could be used to manage and retrieve data from a database. Git could also be used for source control. If the application were developed using Microsoft's technology stack, C#, ASP.NET, and Visual Studio could also be used. Windows also provides a familiar graphical development environment and support for Microsoft development tools. A separate Windows development team would not be needed because the application will use a cross-platform approach with the standard tools of Java, JavaScript, and HTML.	Mobile support could be developed using standard web technologies such as HTML, CSS, and JavaScript. Responsive design frameworks like Bootstrap could help developers create an interface that adapts to different screen sizes. Browser developer tools and mobile device emulators could also help developers test different screen sizes, orientations, and mobile browsers. Developers would need to ensure that buttons, images, timers, and other game features remain easy to use on smaller touchscreen devices. Tools commonly used are as Visual Studio Code, Git, Bootstrap, and browser developer tools, are free. No separate mobile development team would be needed unless the company decides to build a native mobile application.

Recommendations

Operating Platform

I recommend using Linux as the server's operating system because The Gaming Room wants to expand its current application into a web-based version that can serve more users across multiple platforms. Linux can be installed on a server, allowing users to access the gaming application through web browsers on devices running different operating systems, including Windows, macOS, and Linux. Therefore, the company can use a centralized cloud server to host the application rather than requiring a separate server for each client device.

Linux is a popular choice for servers compared with proprietary operating systems such as Windows and macOS. Unless the team plans to use Windows-specific technologies or tools, Linux is the better choice over Windows because its operating system licensing costs are lower and it requires fewer system resources. macOS is less commonly used for server hosting and has hardware and licensing restrictions specific to Apple products. Because Linux is open source, it can help reduce costs by eliminating or lowering operating system licensing fees. Its open-source nature also provides access to a large community that contributes support, updates, and security patches to improve the system.

The company can also choose from many Linux distributions, such as Ubuntu LTS, which provides security updates and maintenance, with additional technical support options available depending on the selected plan. Ubuntu LTS releases occur every two years and receive five years of standard security maintenance. Linux can run on major cloud platforms, including AWS, Microsoft Azure, and Google Cloud, which provide the infrastructure and computing resources needed to host and manage servers. I recommend using Amazon EC2 because it works well with Linux and provides flexible computing resources. Additionally, Linux supports technologies such as containers, load balancing, autoscaling, and virtualization, allowing the company to increase or decrease server resources as the number of users changes. This allows the server's resources to be managed more efficiently.

Operating Systems Architecture

I recommend Linux over macOS and Windows. Windows uses a hybrid kernel called the Windows NT kernel, which has a similar system architecture to Linux and can offer many of the same capabilities. However, Windows has higher licensing costs for cloud-hosted web applications. macOS uses a hybrid kernel called XNU, which is designed for Apple computers and is less commonly used as a server platform. Therefore, I would not recommend macOS or Windows. However, players using Windows, macOS, or Linux devices can connect through a web browser, so each client's operating system handles the display and input, while the Linux server handles the game logic.

Linux uses a monolithic kernel architecture with modular capabilities. The operating system's components operate in kernel space and user space. The kernel manages multiple processes and workloads by allocating resources throughout the operating system. It operates with elevated privileges, allowing it to manage system resources such as CPU processing, memory, files, storage devices, and network interfaces. The kernel provides an interface between applications and the computer's hardware through system calls, which allow applications running on the server to request system resources without directly interacting with the hardware.

User space operates with fewer privileges than kernel space and is where most applications and supporting software run. Components found in user space include system libraries, the initialization

system, package management tools, command-line shells, web server software, and the game application. These components use the kernel's operating system interfaces or system calls to access the resources they need without directly managing the hardware. Separating user space from kernel space also helps protect the operating system by limiting application privileges. If a security issue occurs within a user-space application, this separation can reduce the chance of the issue affecting the kernel.

Storage Management

Storage management can involve many layers, such as the Linux file system, the underlying cloud storage, and PostgreSQL for managing structured data. The file system organizes data into files and directories on a disk. It keeps track of information such as file sizes, ownership, permissions, and locations, and it can help maintain file system consistency after a system outage. Linux servers can use cloud block storage, such as AWS EBS, which acts like an attached hard drive where PostgreSQL data files can be stored. Another option is cloud object storage, such as AWS S3, which can hold large numbers of files, such as images. These images can be stored in object storage for efficient access while remaining separate from the database.

Another recommendation is to use PostgreSQL, a database management system that allows for data storage and retrieval. PostgreSQL pairs well with The Gaming Room because its data is structured. PostgreSQL is also open source, which makes it a cost-effective option, and it works well with the Linux operating system. For example, Linux manages the file system and disk storage where the database files are stored, while PostgreSQL manages the structured data. Since the database stores information persistently on disk, the application can retrieve that information even after a restart.

A relational database can organize structured information into tables and establish relationships between data and objects through primary and foreign keys. It also supports unique constraints that prevent duplicate records and maintain data integrity, such as ensuring that team or player information is not duplicated when multiple users create records in the system simultaneously. The database would allow the application to store, retrieve, and update information about games, teams, players, and other data as users interact with the application. PostgreSQL also supports concurrent database operations by handling multiple user requests at the same time while maintaining data consistency.

Memory Management

The server supports handling multiple requests concurrently, such as loading images, updating game information, processing user requests, and managing active game session information. The Linux operating system uses various memory management techniques to allocate and manage available memory for applications running on the server without exhausting available resources. For example, items such as timers, scores, and the current round's information can be stored in memory for faster access and managed, while static information, such as team and player records, can be saved in PostgreSQL. This also protects the information if the game needs to restart, allowing it to remain available after the restart.

Virtual memory allows each process to have its own virtual address space mapped to physical memory. This allows the game application, PostgreSQL, and the web server to run as separate processes without allowing them to overwrite each other's memory. Within that address space, memory is divided into pages. When applications need memory, Linux can use demand paging to load the required

memory pages instead of requiring the entire application to be loaded into RAM at once. This means that the server does not have to load every image into RAM when the game starts. This allows the application to manage memory more efficiently as the game runs.

Another technique is caching, which allows frequently accessed data to be stored temporarily in memory for faster access. The kernel uses page caching, which allows file data, including images and other data files, to be stored in RAM for quick access. Instead of retrieving images and data files from the disk every time they are needed, the server can access cached data in memory. Therefore, when multiple games are running concurrently and handling multiple requests, these images and files can be accessed more quickly, reducing interruptions to the users' games.

Another technique is the use of swap space in rare cases. When multiple applications compete for physical memory and RAM becomes limited, Linux can use swap space located on an SSD to temporarily hold inactive memory pages that are not currently needed in RAM. This frees up physical memory for other processes to use. Ultimately, it is important to monitor memory usage because, if the system exhausts its available memory, Linux may terminate a process to protect the system from shutting down.

Distributed Systems and Networks

The application can use a distributed system in which the main application runs on a central server while users connect from different devices. Client devices, such as computers running Linux, macOS, or Windows, would send requests to and receive information from the server through a network. A typical workflow involves the user accessing the website through a browser. From there, the browser opens a secure HTTPS connection to a load balancer. The request travels over that secure connection to the load balancer, which forwards the request to an available server. Once the server receives the request, it checks the PostgreSQL database as needed to retrieve or update information. The response then travels back through the load balancer and over the secure HTTPS connection to the user's browser. Because the components depend on one another, a failure in any part of the system could affect the game.

On the server side, the application would run on the Linux operating system and use REST APIs to receive requests from client devices, process the information, and return responses. The server would also communicate with the PostgreSQL database to retrieve and update data as needed and provide the information to client devices. The server would also keep the game state synchronized so that all connected teams and players receive consistent, up-to-date information. Since the game has real-time communication requirements, the server could use WebSockets to maintain an open connection between the server and the client and push real-time updates. This would help reduce interruptions in gameplay.

Network connectivity is essential to the application. If the central server or network connection becomes unavailable, client devices may not be able to receive game updates or join a game session until the connection is restored. Therefore, the application should include measures to handle connection problems, such as internet outages or system failures. The system should be designed to handle temporary network issues through error handling, connection timeouts, and reconnection attempts. It should also display messages that explain the problem and provide next steps for restoring the connection. Additionally, the company should use multiple servers so that if one server fails, the load balancer can redirect requests to another available server.

Security

Throughout the design of the application, security should be built into the Linux server rather than relying solely on the browser. The Linux server should receive regular security updates and use firewalls and appropriate permissions to limit unauthorized access. The server application should validate user input to reduce the risk of attacks.

Linux has built-in protections, such as file permissions, which restrict access based on assigned permissions, and limits what each process can access. These protections can be implemented through security tools such as SELinux or AppArmor, which restrict what applications are allowed to do. The server should communicate over HTTPS, which encrypts information traveling between the user's device and the server. This helps prevent information from being intercepted by unauthorized parties during transmission.

REST APIs should validate incoming requests to ensure that the information being submitted is valid and that users are authorized to perform the requested actions. PostgreSQL can protect sensitive game data by operating on a private network and restricting access to authorized users and applications through role-based access control. User accounts and other sensitive information should be protected through appropriate authentication measures, such as two-factor authentication, and authorization controls, such as access permissions